RET503 COMPUTER VISION AND DEEP LEARNING · MINGGU 4–7

Dari Data ke Model YOLO

CLO-3 · CLO-4 · Program Studi Teknologi Rekayasa Robotika · Politeknik Negeri Batam

Peta Pembelajaran

Empat minggu ini adalah satu alur kerja yang bersambung. Kita mulai dari data mentah, menyiapkannya dalam format yang dipahami YOLO, memperkaya variasinya, melatih model, lalu memperbaikinya sampai layak dipakai. Hasil setiap minggu menjadi bahan minggu berikutnya, sehingga tidak ada tahap yang bisa dilewati.

4

Preparasi data

bounding box, format YOLO/COCO

CLO-3

5

Augmentasi & manajemen

variasi data, train/val/test, versi

CLO-3

6

Melatih YOLO

transfer learning, loss, mAP

CLO-4

7

Tuning & evaluasi

eksperimen sistematis, Pj1

CLO-4

Praktik terpadu di akhir mencakup keempat minggu sekaligus.

Benang merah: transfer learning

Di pertemuan 3, ResNet-18 yang sudah belajar dari ImageNet disesuaikan untuk mengenali baut, mur, dan ring. Kita hanya mengganti bagian keluarannya (head) lalu melatihnya sebentar dengan data kita, dan hasilnya jauh lebih baik daripada melatih dari nol.

	Pertemuan 3	Minggu 4–7
Model sumber	ResNet-18 (ImageNet)	YOLO (COCO)
Tugas	klasifikasi	deteksi objek
Bagian yang disesuaikan	head fc → jumlah kelas	head deteksi → nc di data.yaml

MINGGU 4

Preparasi data YOLO dan COCO

Ringkasan; detail lengkap ada di PPT pertemuan 4

Bounding box dan dua format

Deteksi objek membutuhkan satu label untuk setiap objek: nomor kelas dan kotak. Kotak yang sama bisa ditulis dengan beberapa cara. Format YOLO memakai titik tengah, lebar, dan tinggi yang dibagi ukuran gambar sehingga bernilai 0–1. Format COCO memakai pojok kiri atas, lebar, dan tinggi dalam piksel, dan menyimpan seluruh dataset dalam satu file JSON.

	YOLO	COCO
Satu baris / entri	kelas cx cy w h	{"bbox": [x, y, w, h], "category_id": …}
Satuan	0 – 1	piksel
File	satu .txt per gambar	satu .json untuk semua
Nomor kelas	mulai 0	biasanya mulai 1

Label YOLO memakai titik tengah karena jaringan YOLO juga mengeluarkan kotak dalam bentuk itu, sehingga tebakan dan label bisa dibandingkan langsung saat pelatihan.

Struktur folder dan data.yaml

```
dataset_v1/
├── data.yaml
├── images/
│   ├── train/
│   ├── val/
│   └── test/
└── labels/
    ├── train/
    ├── val/
    └── test/
```

```
names:
  0: person
  1: car
  2: cup
```

YOLO menemukan file label dengan mengganti /images/ menjadi /labels/ pada jalur gambar. Karena itu nama folder harus tepat, dan setiap gambar harus punya file label bernama sama.

Mengapa COCO, dan apakah harus COCO?

ImageNet dirancang untuk klasifikasi: satu label untuk satu gambar. COCO dirancang untuk deteksi: banyak objek per gambar, masing-masing diberi kotak, dalam adegan sehari-hari yang ramai. Model YOLO pretrained dilatih dengan COCO, jadi untuk deteksi kita memakai COCO.

COCO tidak wajib. Di praktik ini COCO dipakai karena anotasinya sudah tersedia dan berkualitas, sehingga mahasiswa bisa fokus pada alur kerjanya. Untuk projek nyata, data ditentukan oleh objek yang ingin dideteksi. Jika objeknya tidak ada di COCO, misalnya “jalan berlubang”, dibutuhkan dataset lain atau data sendiri, dengan alur program yang sama mulai dari pemeriksaan format.

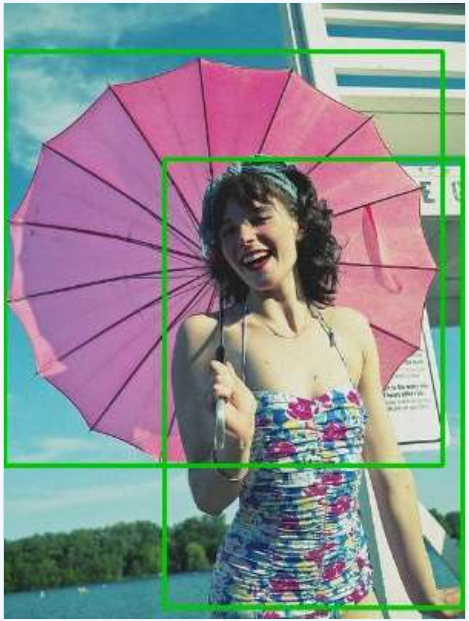
COCO: 330 ribu gambar, 200 ribu beranotasi, 80 kelas. train2017 118.287 gambar, val2017 5.000 gambar.

MINGGU 5

Augmentasi dan manajemen dataset

Membuat data cukup bervariasi dan evaluasi tetap jujur

Mengapa perlu augmentasi?

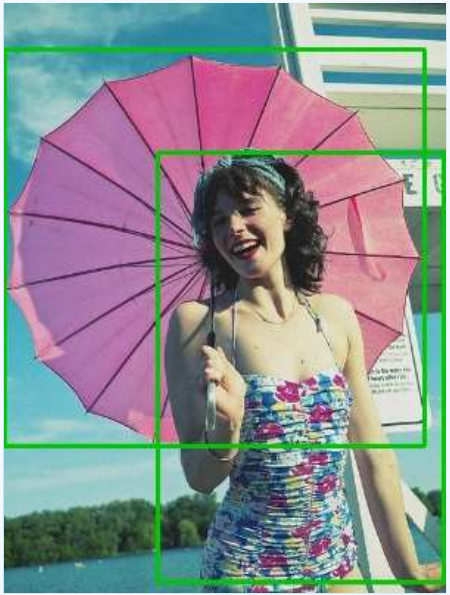


Gambar asli COCO dengan kotak

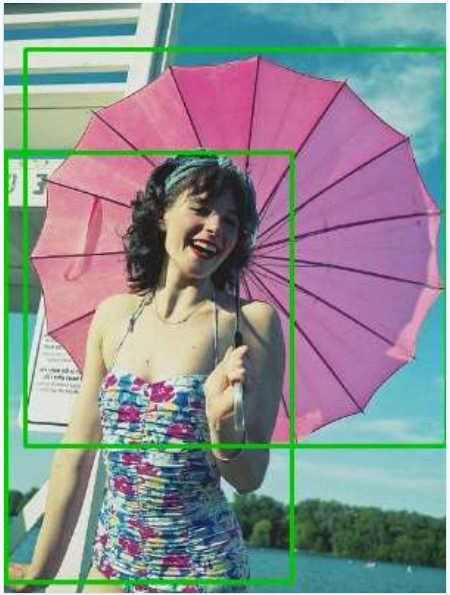
Dataset projek biasanya hanya ratusan gambar, dan sering diambil dalam kondisi yang mirip: cahaya yang sama, sudut yang sama, latar yang sama. Model yang hanya melihat variasi sesempit itu mudah menghafal dan gagal saat kondisinya sedikit berbeda.

Augmentasi membuat variasi baru dari gambar yang sudah ada, misalnya dibalik, digeser, diperbesar, atau diubah kecerahannya, tanpa harus memotret ulang. Tujuannya bukan sekadar memperbanyak gambar, tetapi meniru variasi yang benar-benar akan ditemui model saat dipakai. Ada dua kelompok besar: augmentasi geometri yang mengubah posisi piksel, dan augmentasi warna yang hanya mengubah nilai piksel.

Augmentasi geometri: kotak ikut berubah



Asli



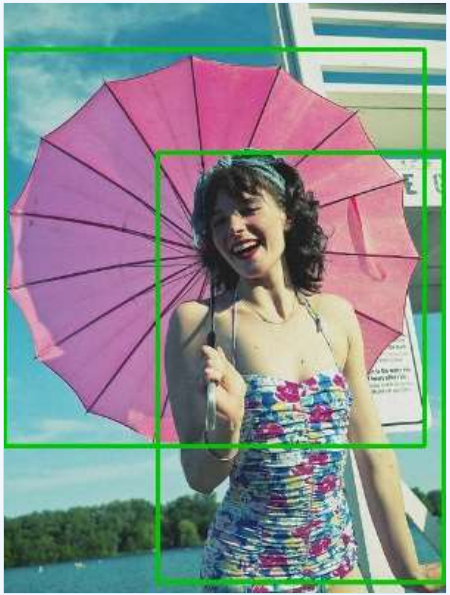
Dibalik kiri-kanan

Dibuat dari gambar COCO 000000000036; kotak dihitung ulang dengan rumus di atas

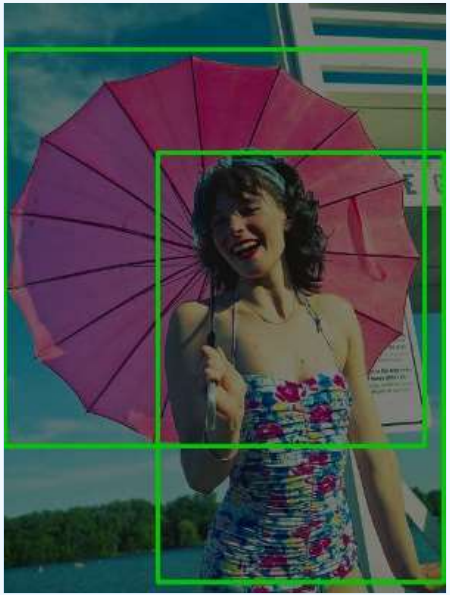
Saat gambar dibalik, digeser, atau diputar, posisi objek ikut berubah, jadi kotaknya harus dihitung ulang. Untuk flip kiri-kanan, rumusnya sederhana: titik tengah x yang baru adalah 1 − cx, sedangkan cy, lebar, dan tinggi tetap. Perhatikan pada gambar: kotak payung dan orang ikut berpindah ke sisi seberang.

```
# flip kiri-kanan pada label YOLO
cx_baru = 1 - cx
```

Augmentasi warna: kotak tetap



Asli

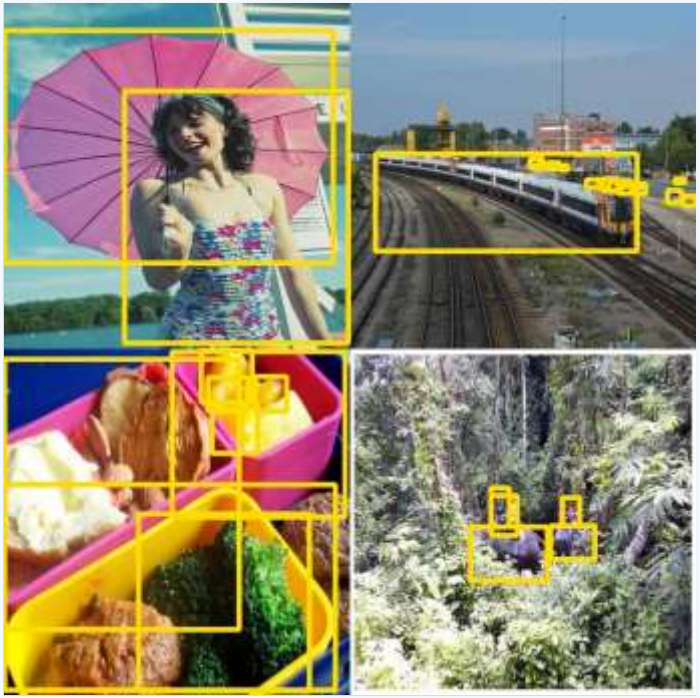


Lebih redup dan jenuh

Augmentasi warna mengubah kecerahan, saturasi, atau corak warna, tetapi tidak memindahkan piksel. Karena itu kotaknya sama persis dengan gambar asli. Pada gambar kanan, kecerahan diturunkan sekitar 45% dan saturasi dinaikkan, meniru foto yang diambil saat cahaya kurang.

Augmentasi ini membantu model tetap mengenali objek walaupun pencahayaan berubah, masalah yang sangat umum di lapangan.

Mosaic: empat gambar menjadi satu



Mosaic 2×2 dari 4 gambar COCO, 31 kotak ikut diskalakan

Mosaic menggabungkan empat gambar menjadi satu gambar latih. Setiap gambar diperkecil, lalu kotaknya ikut diskalakan dan digeser sesuai posisinya. Hasilnya, satu gambar latih berisi banyak objek dengan ukuran dan latar yang beragam.

Mosaic sangat membantu untuk objek kecil dan dataset kecil, karena model melihat lebih banyak kombinasi objek dan latar dalam setiap langkah pelatihan. Di Ultralytics, mosaic aktif secara bawaan dan biasanya dimatikan otomatis pada beberapa epoch terakhir, agar model sempat belajar dari gambar utuh. Contoh di samping adalah versi sederhana 2×2; Ultralytics memakai titik potong acak.

Augmentasi otomatis di Ultralytics

Ultralytics melakukan augmentasi secara otomatis setiap kali gambar dimuat saat pelatihan (online), jadi kita tidak perlu membuat ribuan file gambar baru. Kita cukup mengatur kekuatan setiap jenis augmentasi lewat parameter berikut.

Parameter	Bawaan umum	Efek	Kapan diubah
fliplr	0.5	balik kiri-kanan, peluang 50%	set 0 jika arah kiri-kanan membedakan kelas
flipud	0.0	balik atas-bawah	naikkan hanya jika objek bisa terbalik
degrees	0.0	rotasi acak	naikkan sedikit jika kamera bisa miring
scale	0.5	perbesar/perkecil ±50%	turunkan jika jarak objek selalu tetap
hsv_h, hsv_s, hsv_v	0.015, 0.7, 0.4	corak, saturasi, kecerahan	turunkan hsv_h jika warna membedakan kelas
mosaic	1.0	gabungkan 4 gambar	biasanya dibiarkan aktif

Augmentasi yang merusak makna

Tidak semua augmentasi aman. Prinsipnya: augmentasi hanya boleh menghasilkan gambar yang mungkin ditemui model di dunia nyata. Jika tidak, model belajar dari contoh yang mustahil, dan akurasinya bisa justru turun.

Flip atas-bawah
Hindari jika objek yang selalu tegak, seperti orang atau botol berdiri, tidak pernah terlihat terbalik

Rotasi besar
Hindari jika kamera yang selalu sejajar lantai tidak akan melihat objek miring 90°

Ubah warna kuat
Hindari jika jika kelas dibedakan oleh warna, misalnya kabel merah dan biru

Flip kiri-kanan
Hindari jika jika arah membedakan kelas, misalnya rambu belok kiri dan kanan

Train, val, dan test

Dataset dibagi menjadi tiga bagian dengan peran berbeda. Data train dipakai untuk belajar. Data val dipakai selama pengembangan untuk memilih model dan pengaturan terbaik. Data test disimpan dan baru dipakai sekali di akhir, untuk melaporkan kinerja dengan jujur. Augmentasi hanya diterapkan pada data train.

Bagian	Proporsi	Contoh asli
train	70%	46 gambar
val	20%	13 gambar
test	10%	7 gambar

Data asli: subset COCO (coco128), keluaran program minggu 4

Foto kembar harus dihapus sebelum data dibagi. Jika satu foto ada di train dan kembarannya di val, nilai val menjadi terlalu tinggi (data leakage), seperti yang dibahas di minggu 3.

Ketidakseimbangan kelas



Data asli: subset COCO (coco128), keluaran program minggu 4

Pada subset COCO yang kita ambil, kelas person punya 187 kotak, sedangkan car dan cup masing-masing sekitar 30. COCO memang didominasi foto berisi orang, dan satu foto keramaian bisa berisi belasan orang.

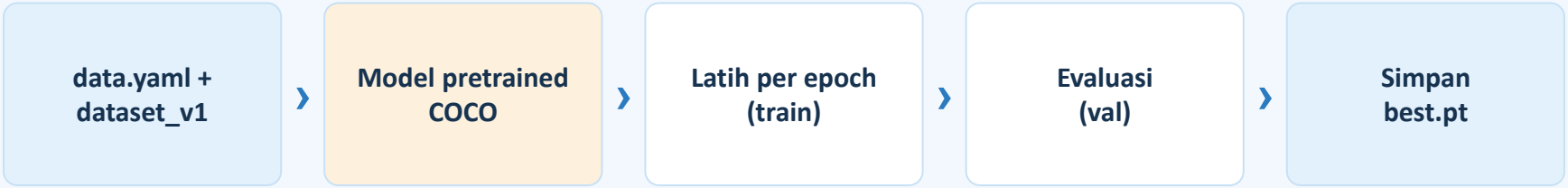
Akibatnya, model akan lebih sering melihat person dan cenderung lebih pandai mengenalinya. Augmentasi saja tidak menyelesaikan masalah ini, karena augmentasi memperbanyak semua kelas secara merata. Cara yang lebih efektif adalah menambah data kelas yang sedikit, lalu memeriksa hasil per kelas, bukan hanya rata-ratanya.

MINGGU 6

Melatih YOLO

Transfer learning, loss, dan metrik deteksi

Alur pelatihan YOLO



Pelatihan YOLO dijalankan dengan satu perintah, tetapi di dalamnya ada beberapa langkah. Ultralytics membaca data.yaml untuk mengetahui lokasi data dan jumlah kelas, lalu memuat model pretrained COCO. Setelah itu model dilatih berulang kali. Satu epoch berarti seluruh data train sudah dilewati sekali.

Setiap selesai satu epoch, model dievaluasi dengan data val. Bobot dengan nilai val terbaik disimpan sebagai best.pt, dan bobot epoch terakhir sebagai last.pt. Jadi model yang kita pakai nanti adalah model terbaik selama pelatihan, bukan otomatis model terakhir.

Transfer learning di YOLO

Saat model pretrained COCO (80 kelas) dilatih dengan data.yaml berisi 3 kelas, Ultralytics membuat ulang bagian keluaran kelas menjadi 3, lalu memindahkan semua bobot lain yang ukurannya cocok dari model pretrained. Backbone yang sudah pandai mengenali tepi, tekstur, dan bentuk dipakai ulang, sehingga model cukup belajar membedakan kelas-kelas kita.

	Pretrained (yolo26n.pt)	Dari nol (yolo26n.yaml)
Bobot awal	hasil belajar dari COCO	acak
Yang dipelajari	menyesuaikan dengan kelas kita	semuanya dari awal
Data yang dibutuhkan	ratusan gambar sudah cukup	jauh lebih banyak
Pola yang diharapkan	mAP naik cepat dan tinggi	mAP naik lambat dan rendah

Pola ini yang dibuktikan di praktik terpadu (E1 vs E2), sama seperti feature/partial vs scratch di minggu 3.

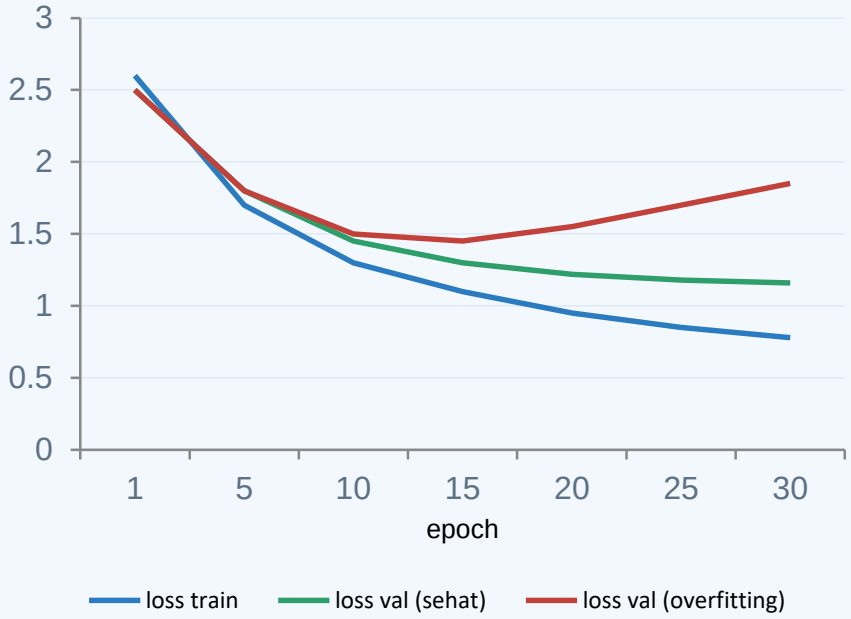
Perintah dan argumen pelatihan

```
yolo detect train data=dataset_v1/data.yaml model=yolo26n.pt epochs=30 imgsz=416 batch=16
```

Argumen	Arti	Pertimbangan
data	lokasi data.yaml	harus berisi path, train, val, names
model	titik awal (.pt = pretrained, .yaml = dari nol)	inilah yang menentukan ada transfer learning atau tidak
epochs	berapa kali data train dilewati	terlalu sedikit = belum belajar; terlalu banyak = overfitting
imgsz	ukuran gambar input	lebih besar = objek kecil lebih terlihat, tetapi lebih lambat
batch	jumlah gambar per langkah	turunkan jika memori GPU tidak cukup

Argumen model adalah yang paling penting untuk dipahami: berkas .pt berarti mulai dari bobot pretrained, sedangkan .yaml berarti hanya arsitekturnya, tanpa pengetahuan apa pun.

Loss dan cara membaca kurvanya



Ilustrasi pola kurva (bukan hasil pelatihan)

Loss mengukur seberapa jauh tebakan model dari label.

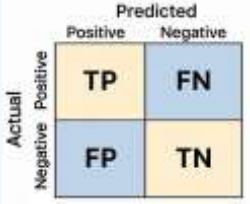
Pada YOLO, loss terdiri dari beberapa komponen, misalnya box loss untuk ketepatan posisi kotak dan cls loss untuk ketepatan kelas. Nama dan jumlah komponennya bisa berbeda antarversi YOLO.

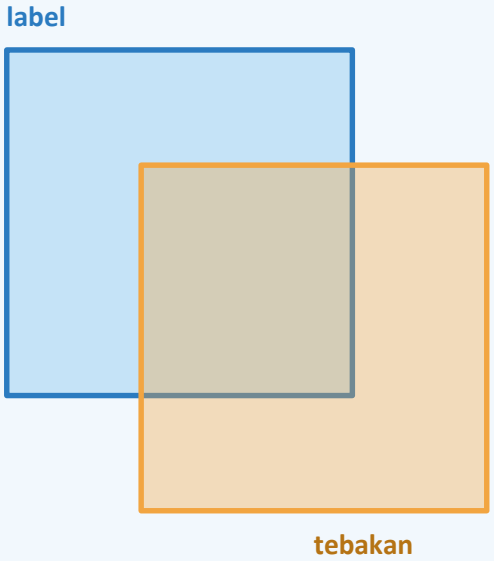
Yang penting adalah membaca polanya. Jika loss train dan loss val sama-sama turun, model belajar dengan sehat.

Jika loss train terus turun tetapi loss val berbalik naik, model mulai menghafal data train (overfitting).

Model terbaik ada di sekitar titik terendah loss val, dan itulah yang disimpan sebagai best.pt.

Kapan sebuah deteksi dianggap benar?



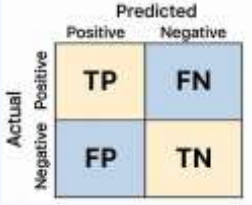


Untuk menilai model, setiap tebakan dibandingkan dengan label. Kedekatan dua kotak diukur dengan IoU (Intersection over Union): luas bagian yang tumpang tindih dibagi luas gabungannya, bernilai 0 sampai 1.

Hasil	Arti
TP (true positive)	tebakan cocok dengan label: kelas benar dan IoU ≥ ambang, misalnya 0,5
FP (false positive)	tebakan tanpa pasangan label: objek palsu atau kotak meleset
FN (false negative)	label tanpa tebakan: objek yang terlewat

```
IoU = luas irisan ÷ luas gabungan
```

Precision dan recall



`Precision = TP / (TP + FP)`
Dari semua kotak yang ditebak, berapa yang benar?

`Recall = TP / (TP + FN)`
Dari semua objek yang ada, berapa yang ditemukan?

Contoh: dalam data val ada 12 objek. Model membuat 10 tebakan, 8 di antaranya benar (TP), 2 salah (FP), dan 4 objek terlewat (FN). Precision = 8 / (8 + 2) = 0,80, artinya tebakannya jarang salah. Recall = 8 / (8 + 4) = 0,67, artinya sepertiga objek masih terlewat.

Keduanya saling tarik-menarik. Menurunkan ambang keyakinan membuat model menebak lebih banyak: recall naik, tetapi precision turun. Untuk robot yang harus menghindari tabrakan, recall tinggi biasanya lebih penting; untuk sistem yang menghitung barang, precision tinggi lebih penting.

AP dan mAP: satu angka untuk semuanya

Precision dan recall berubah-ubah tergantung ambang keyakinan. Agar bisa dibandingkan dengan satu angka, precision digambar terhadap recall untuk semua ambang (kurva precision-recall), lalu dihitung luas di bawah kurvanya. Luas ini disebut AP (average precision), dan dihitung untuk setiap kelas. Rata-rata AP semua kelas disebut mAP.

Metrik	Arti	Kapan dipakai
mAP50	mAP dengan ambang IoU 0,50	longgar: cukup jika kotak kira-kira tepat
mAP50-95	rata-rata mAP pada 10 ambang IoU: 0,50; 0,55; …; 0,95	ketat: kotak harus benar-benar pas; ukuran utama COCO

Di praktik terpadu, model terbaik dipilih berdasarkan mAP50-95 di data val. Pembahasan evaluasi yang lebih mendalam, termasuk confusion matrix dan kecepatan inferensi, ada di minggu 8.

File hasil pelatihan

```
runs/detect/E1_pretrained/
├── weights/
│   ├── best.pt
│   └── last.pt
├── results.csv
├── results.png
├── confusion_matrix.png
├── (kurva PR, F1, dll.)
└── args.yaml
```

Setiap pelatihan menghasilkan satu folder. best.pt adalah model dengan nilai val terbaik, yang dipakai untuk prediksi dan laporan. results.csv berisi angka loss dan metrik setiap epoch, dan results.png adalah grafiknya.

confusion_matrix.png menunjukkan kelas mana yang sering tertukar atau terlewat. args.yaml menyimpan semua pengaturan pelatihan, sehingga eksperimen bisa diulang persis sama. Biasakan menyimpan seluruh folder ini untuk laporan, bukan hanya angka mAP-nya.

MINGGU 7

Tuning dan evaluasi akhir

Memperbaiki model secara sistematis

Prinsip tuning: satu perubahan per eksperimen

Setelah punya baseline, godaan terbesarnya adalah mengubah banyak hal sekaligus. Masalahnya, jika hasil naik, kita tidak tahu perubahan mana yang membantu; jika turun, kita tidak tahu mana yang merusak.

Karena itu tuning dilakukan seperti eksperimen ilmiah: ubah satu hal, pertahankan semua pengaturan lain, latih ulang, lalu bandingkan dengan baseline. Jika hasilnya membaik, perubahan itu dipertahankan dan menjadi baseline baru. Jika tidak, perubahan itu dibatalkan. Prinsip ini sama dengan eksperimen tiga mode di minggu 3, ketika satu-satunya perbedaan antar-mode adalah strategi pembekuannya.



Apa saja yang bisa diubah?

Yang diubah	Contoh	Dicoba jika
Lama pelatihan	epochs=60 patience=20	kurva mAP val masih naik di epoch terakhir
Ukuran input	imgsz=640	objek kecil sering terlewat
Ukuran model	model=yolo26s.pt	akurasi kurang dan komputasi masih longgar
Learning rate	optimizer=SGD lr0=0.005	loss naik-turun tidak stabil
Augmentasi	degrees=10, hsv_v=0.6	model gagal pada kondisi miring atau redup
Data	dataset_v2 (tambah kelas minoritas)	confusion matrix menunjukkan kelas tertentu lemah

Catatan learning rate: dengan pengaturan bawaan optimizer=auto, Ultralytics memilih optimizer dan learning rate sendiri dan mengabaikan lr0. Agar lr0 benar-benar dipakai, tetapkan optimizer secara eksplisit. Dalam praktik, perubahan pada data sering memberi peningkatan terbesar.

Mencatat setiap eksperimen

Eksperimen	Perubahan	Dataset	mAP50	mAP50-95	ms/gambar	Kesimpulan
E1_pretrained	– (baseline)	v1	…	…	…	titik pembanding
E2_scratch	bobot acak	v1	…	…	…	manfaat transfer learning
E3_tuning	imgsz=640	v1	…	…	…	dipertahankan / dibatalkan

Setiap eksperimen dicatat dalam satu baris: apa yang diubah, versi dataset, hasilnya, dan kesimpulannya. Tanpa catatan, setelah beberapa hari kita tidak ingat lagi pengaturan mana yang menghasilkan model terbaik. Program praktik terpadu membuat tabel seperti ini secara otomatis untuk tiga eksperimen pertama.

Angka diisi dari hasil pelatihan masing-masing kelompok.

Memilih model dan menguji di data test

Model dipilih berdasarkan mAP50-95 di data val. Setelah model terpilih, barulah data test dipakai, dan hanya sekali. Angka test inilah yang dilaporkan sebagai kinerja akhir. Jika data test dipakai berulang kali untuk memilih, test berubah menjadi val kedua dan angkanya tidak lagi jujur.

Akurasi bukan satu-satunya pertimbangan. Model yang lebih besar atau dengan imgsz lebih besar biasanya lebih akurat, tetapi lebih lambat. Bandingkan waktu inferensi per gambar dengan anggaran latensi di dokumen desain minggu 3. Model terbaik adalah yang cukup akurat dan masih memenuhi target FPS di perangkat yang dipakai.

```
yolo detect val model=hasil_praktik/E3_tuning/weights/best.pt data=dataset_v1/data.yaml split=test
```

Pj1: Perception System Concept and Design

Minggu 7 ditutup dengan Pj1, yang menurut RPS menggabungkan seluruh materi minggu 3–7 untuk projek kelompok. Pj1 menunjukkan bahwa kelompok bisa merancang sistem persepsi dari tujuan projek sampai model yang terukur.

1. Menentukan lingkup persepsi dari misi projek
2. Menetapkan target kinerja: mAP, FPS, dan latensi
3. Merancang pipeline sesuai kamera, posisi, dan perangkat
4. Mengambil dan menganotasi dataset format YOLO/COCO
5. Melatih dan melakukan tuning baseline YOLO dengan log eksperimen

PRAKTIK TERPADU

Dari COCO ke detektor YOLO

Satu praktik yang mencakup minggu 4–7

Gambaran praktik terpadu

Tahap	Minggu	Kegiatan	Program
1	4	Ambil subset COCO (3 kelas pilihan kelompok), cek, konversi	00, 01, 02, 03
2	5	Bagi train/val/test, buat data.yaml, tentukan augmentasi	05
3	6	Latih E1 pretrained dan E2 dari nol	06
4	7	Latih E3 dengan satu perubahan	06
5	6–7	Pilih model dengan val, uji sekali dengan test	06
6	4–7	Laporan: tabel, grafik, kesimpulan	–

Praktik ini satu alur dari awal sampai akhir. Tahap 3–5 dijalankan oleh satu program sekaligus. Pelatihan butuh waktu, jadi gunakan GPU di lab atau Google Colab, dan bagi praktik ke beberapa sesi lab dengan checkpoint per tahap.

Tahap 1–2: menyiapkan data

```
python 00_ambil_subset_coco.py --kelas <3 kelas pilihan> --jumlah 200
python 01_cek_anotasi.py dataset_yolo
python 02_yolo_ke_coco.py dataset_yolo dataset_yolo/annotations_coco.json
python 03_coco_ke_yolo.py dataset_yolo/annotations_coco.json hasil_balik
python 05_split_dataset_yolo.py dataset_yolo dataset_v1
```

Tahap 1 menghasilkan dataset_yolo berisi gambar dan label YOLO untuk tiga kelas pilihan kelompok, lalu memeriksa format dan menguji konversi dua arah. Tahap 2 membaginya menjadi dataset_v1 dengan train, val, dan test, lengkap dengan data.yaml.

Setiap kelompok wajib memilih kombinasi kelas yang berbeda. Pilih minimal satu kelas yang jumlahnya sedikit di COCO, agar pengaruh ketidakseimbangan kelas terlihat di hasil per kelas.

Tahap 3–5: program 06_latih_bandingkan.py

```
python 06_latih_bandingkan.py --nim 4222101001 --nama "Budi Santoso"
```

E1 pretrained
`yolo26n.pt`
baseline, dengan transfer learning dari COCO

E2 scratch
`yolo26n.yaml`
arsitektur sama, bobot acak, tanpa transfer learning

E3 tuning
`yolo26n.pt + satu perubahan`
default imgsz=640; ubah dengan --ubah kunci=nilai

Ketiga eksperimen memakai data, epoch, ukuran input, dan seed yang sama, sehingga perbedaan hasil hanya disebabkan oleh satu faktor. Perbandingan E1 dengan E2 menunjukkan manfaat transfer learning. Perbandingan E1 dengan E3 menunjukkan hasil satu langkah tuning. Program lalu memilih model dengan mAP50-95 val tertinggi dan mengujinya sekali di data test.

Keluaran program 06 dan cara membacanya

```
NIM / Nama    : ... / ...
Dataset       : dataset_v1  (sidik jari label: ...)
Eksperimen     Perubahan   mAP50 mAP50-95   P    R   ms menit
E1_pretrained  -            ...    ...     ...  ... ...  ...
E2_scratch     -            ...    ...     ...  ... ...  ...
E3_tuning      imgsz=640    ...    ...     ...  ... ...  ...
Model terpilih (mAP50-95 val tertinggi): ...
Hasil di data TEST (sekali saja)       : ...
```

Format keluaran; disimpan juga ke hasil_praktik_<NIM>.txt dan .csv

Pola yang diharapkan: E1 jauh lebih tinggi daripada E2 pada epoch yang sama, karena E1 membawa pengetahuan dari COCO. E3 bisa lebih baik atau lebih buruk dari E1; keduanya sama-sama hasil yang sah, asalkan dijelaskan.

Identitas, waktu, komputer, dan sidik jari dataset dicetak otomatis, sehingga hasil yang disalin dari kelompok lain mudah dikenali.

Yang dikumpulkan dan penilaiannya

Bagian	Yang dikumpulkan	Asesmen RPS
Tahap 1–2	Data projek mahasiswa	P3 (CLO-3)
Tahap 3	hasil E1 dan E3, results.png kedua eksperimen	P4 (CLO-4)
Tahap 4–6	hasil_praktik_<NIM>.txt/.csv, analisis dan hasil test (Read.Me)	Pj1 (CLO-4)

Walaupun praktiknya hanya satu, setiap tahapnya menghasilkan bukti untuk asesmen yang berbeda di RPS. Untuk Pj1, alur yang sama sebaiknya diulang dengan data projek kelompok, karena Pj1 meminta dataset dan target kinerja dari misi projek sendiri.

SETELAH INI · UTS, LALU MINGGU 8

Evaluasi kinerja deteksi secara mendalam

Ultralytics Academy, "Prepare a Custom Dataset", kursus Train your first YOLO model.

Ultralytics Docs, "COCO Dataset".

Holzer, R., "YOLO – object detection", OpenCV tutorial, 2019.

Redmon, J. dkk., "You Only Look Once", CVPR 2016.

Lin, T.-Y. dkk., "Microsoft COCO: Common Objects in Context", ECCV 2014.

Data contoh: Ultralytics coco128 (gambar dan anotasi COCO).